NUTM VITA-NOVA
Product Requirements Document (PRD)

Version 1.0 — Sections 1–52
Master Requirements & Implementation Specification

Status: V1 requirements/design baseline locked through Section 52.

Purpose: Define the functional, technical, workflow, security, data, UI, testing, deployment and acceptance requirements for a university clinic mini Electronic Medical Record (EMR).

Section 1–52 — Executive Summary

NUTM VITA-NOVA is a role-based mini EMR for a university clinic serving students, staff and staff dependants. It replaces the paper workflow with a longitudinal digital record while preserving professional accountability, least-privilege access and a practical V1 scope. The system is a modular Flask monolith backed by PostgreSQL, with the encounter as the central workflow object. Sections 1–52 have been agreed and locked; major V2 features are explicitly deferred.

Section 2 — Product Context & Problem

- Paper-based records can be lost, difficult to retrieve and difficult to use longitudinally.
- Paper processes consume files, cards, paper and writing materials.
- Paper records make operational reporting and analytics difficult.
- V1 focuses on workflow efficiency and reliable digital record retrieval rather than solving every possible digital-health problem.

Section 3 — Users, Population & Clinic Context

- Population: students, staff and staff dependants.
- No payment workflow in V1.
- Routine outpatient care dominates; emergencies are rare/sporadic.
- Functional areas: front desk, nursing, doctors, laboratory and pharmacy.
- V1 is designed to eliminate the need for dedicated records officers.

Section 4 — Roles & Access Philosophy

- Desk Officer — demographic/administrative information only.
- Nurse — broad clinical-history visibility; creates/edits own nursing records.
- Doctor — broad clinical access; creates/amends own doctor records.
- Laboratory Scientist — laboratory-relevant information only.
- Pharmacist — pharmacy-relevant information and relevant diagnoses/prescriptions.
- Administrator — maximum system visibility and management, but no unrestricted clinical editing.

Section 5 — Authentication & User Accounts

- Individual staff accounts; no shared departmental accounts.
- Phone number is the username; no separate username field.
- Administrator creates accounts, assigns roles and generates default passwords.
- Passwords are securely hashed; administrators cannot view actual passwords.
- Admin can reset passwords, activate/deactivate accounts and view last login.
- Deactivated users cannot log in; historical records remain linked.
- One-hour inactivity timeout and explicit logout.
- Generic login errors, CSRF protection, secure sessions and server-side authorization.

Section 6 — Patient Identity & Categories

- Each person receiving care has a unique patient record and Clinic ID.
- Categories: STUDENT, STAFF, STAFF_DEPENDANT.
- Clinic ID is system-generated and cannot be manually selected.
- Student IDs and Staff IDs are unique when present.
- Dependants have their own Clinic IDs and longitudinal records, linked to the staff beneficiary.

- DOB is stored; age is calculated dynamically.
- No patient photo, allergies, blood group or genotype in V1.

Section 7 — Patient Registration & Demographics

- Desk can register, search, edit demographics, register dependants, correct mistakes and deactivate records.
- Student: full name, student ID, DOB, sex, phone, department/faculty, level/year, address, next of kin.
- Staff: full name, staff ID, DOB, sex, phone, occupation, address, next of kin.
- Dependant: full name, DOB, sex, phone, address, next of kin, linked staff beneficiary.
- Non-applicable fields remain NULL.
- V1 has uniqueness constraints but no advanced duplicate-prevention engine.
- Demographic edits have no dedicated amendment history in V1.
- Patient records are deactivated rather than physically deleted.

Section 8 — Encounter Model

- One patient can have many encounters; only Desk Officers create encounters.
- No queue number in V1.
- One encounter contains multiple departmental activities.
- Fields: encounter_id, patient_id, encounter_type, encounter_date, arrival_time, created_by, created_at, status.
- Types: New Patient, Follow-up, Nursing-only, Lab-only, Medication-related, Emergency.
- Status: OPEN/CLOSED.
- Doctor final submission closes the encounter; late lab finalization does not reopen it.
- New consultation after closure requires a new encounter.

Section 9 — Standard Clinical Workflow

Patient arrives → Desk search/register → Create encounter → Nursing → Doctor → Laboratory and/or Prescription → Pharmacy → Longitudinal record.

- Workflow is visible but not rigidly enforced as a department lock.
- Late finalized lab results attach to a closed encounter.
- Cross-department indicators reflect actual database state.

Section 10 — Nursing V1 Requirements

- Multiple nursing records per encounter.
- Vitals: temperature, systolic/diastolic BP, pulse, respiratory rate, SpO■.
- Anthropometry: weight, height and automatic BMI; no BMI classification.
- Free-text nursing notes with automatic author/date/time.
- Only original submitting nurse may amend; mandatory reason; original preserved.
- SAVE = draft; SUBMIT = finalized.
- Medication administration supports structured/free-text entry and statuses Given, Not Given, Refused, Not Available, Other.

Section 11 — Doctor V1 Requirements

- Doctor sees patient identity, current nursing assessment and longitudinal clinical history.
- Choose Free-text or Structured documentation.

- Structured history: complaints, HPC, PMH, drug history, family history, ROS.
- Hybrid examination: general, cardiovascular, respiratory, abdominal, neurological, musculoskeletal, other; relevant nursing observations are pulled in.
- Assessment: primary, multiple secondary and optional differential diagnoses.
- Plan/management free text.
- SAVE keeps OPEN; FINAL SUBMISSION closes encounter and automatically attaches electronic signature.
- Only original doctor can amend; mandatory reason and complete version history.
- Diagnoses remain in doctor documentation; no diagnosis table in V1.

Section 12 — Laboratory V1 Requirements

- Searchable investigation catalogue or free-type investigation.
- Each investigation gets a unique request ID, e.g. LAB-2026-000184.
- One investigation request has at most one laboratory result.
- Optional clinical information is manually supplied by doctor.
- No specimen workflow in V1.
- Result is free text with NORMAL/ABNORMAL/CRITICAL flag.
- SAVE = draft; FINALIZE = official result.
- Draft results are visible only to lab scientists; finalized results to authorized clinical users.
- Only original lab scientist may amend; original and corrected versions preserved.
- Historical requests retain their investigation name after catalogue deactivation.

Section 13 — Pharmacy & Prescription V1 Requirements

- Structured prescription fields are all optional: medication, dose, formulation, route, frequency, duration, instructions.
- Multiple prescription items allowed; status DRAFT or FINALIZED.
- Only finalized prescriptions reach Pharmacy.
- Pharmacist cannot modify prescriptions.
- Clarification uses an attached doctor-pharmacist communication thread.
- One prescription has at most one dispensing record.
- No inventory, stock, partial dispensing or automated drug-interaction engine in V1.

Section 14 — Administrator Requirements

- Manage users, roles, password resets, activation/deactivation and investigation catalogue.
- View system records and amendment history.
- View patient demographics but not edit them.
- Full visibility does not imply unrestricted clinical editing.
- Catalogue supports add/view/deactivate; historical requests remain intact.

Section 15 — Search & Longitudinal Record

- Patient search: Clinic ID, Student ID, Staff ID, name and phone number; name supports partial matching.
- Lab search: request number, patient, investigation.
- Pharmacy search: prescription, patient.
- Previous encounters remain visible in longitudinal history.
- Encounter activities display most recent first.

- Filters: All, Doctor, Nursing, Laboratory, Prescription, Pharmacy, Date.
- Search results and views are role-restricted.

Section 16 — Notifications & Status Indicators

- Simple in-app status indicators only; no SMS/email/WhatsApp/push.
- Nursing: Pending/Draft/Submitted.
- Laboratory: Pending/Draft/Result Available.
- Prescription: Draft/Ready for Pharmacy.
- Pharmacy: Pending Dispensing/Dispensed.
- Status reflects actual database state; no complex global notification inbox.

Section 17 — UI Design System

- Consistent top bar, role-specific sidebar and main content area.
- Top bar shows NUTM VITA-NOVA, user, role and Logout.
- Role-specific dashboards.
- Consistent patient and encounter headers.
- Forms grouped into logical sections.
- Clear state-changing actions and confirmation where required.
- Responsive for desktop/laptop/tablet; mobile is not primary V1 target.
- Clean professional healthcare aesthetic.

Section 18 — Error Handling & Validation

- Client-side validation for immediate feedback plus authoritative server-side validation.
- Validate required fields, types, uniqueness, category rules and workflow states.
- Database errors become friendly messages; raw SQL/stack traces are never shown.
- Unauthorized actions return safe access-denied responses.
- Unexpected errors produce safe user messages and technical server logs.
- Success confirmations are temporary toasts/alerts that automatically disappear after 2 seconds.
- Important finalization actions require confirmation.

Section 19 — Security Architecture

- Least-privilege RBAC across six roles.
- Server-side authorization for protected workflows.
- Flask-Login, Werkzeug password hashing and secure sessions.
- One-hour inactivity timeout and CSRF protection.
- Parameterized database operations.
- Foreign keys, unique constraints and controlled statuses.
- No shared accounts and no reliance on hidden UI controls.
- V2: 2FA, break-glass, field encryption and advanced monitoring.

Section 20 — Audit & Versioning

- Clinical records are never silently overwritten/deleted.
- Versioning applies to nursing, doctor and laboratory records.

- RECORD_AMENDMENTS stores record type/id, author/time, reason, previous_content JSONB and new_content JSONB.
- Only original author may amend; multiple amendments allowed.
- Demographic edits are the V1 exception without dedicated versioning.

Section 21 — Database Design — V1 Tables

#	Table
1	roles
2	users
3	patients
4	encounters
5	nursing_documentations
6	medication_administrations
7	doctor_documentations
8	investigation_requests
9	laboratory_results
10	prescriptions
11	prescription_items
12	pharmacy_communications
13	pharmacy_dispensings
14	record_amendments
15	investigations

Diagnoses remain in doctor_documentations in V1.

Section 22 — Core ERD Relationships

ROLES 1 ■■■< USERS
PATIENTS 1 ■■■< ENCOUNTERS
ENCOUNTER 1 ■■■< NURSING_DOCUMENTATIONS
ENCOUNTER 1 ■■■< MEDICATION_ADMINISTRATIONS
ENCOUNTER 1 ■■■< DOCTOR_DOCUMENTATIONS
ENCOUNTER 1 ■■■< INVESTIGATION_REQUESTS ■■■ 0..1 LABORATORY_RESULT
ENCOUNTER 1 ■■■< PRESCRIPTIONS ■■■< PRESCRIPTION_ITEMS
PRESCRIPTION 1 ■■■< PHARMACY_COMMUNICATIONS
PRESCRIPTION 1 ■■■ 0..1 PHARMACY_DISPENSING
USERS 1 ■■■< RECORD_AMENDMENTS
PATIENTS 1 ■■■< PATIENTS (beneficiary → staff patient)
INVESTIGATIONS = reference catalogue only.

Section 23 — Database Integrity & Constraints

- PK on all 15 tables; FK on defined relationships.
- Unique Clinic ID, phone number, Student ID, Staff ID and lab request number.
- Unique investigation-result and prescription-dispensing relationships.
- NOT NULL, controlled statuses and PostgreSQL CHECK constraints.
- Category validation in Python.
- No clinical cascade deletes; ON DELETE RESTRICT; ON UPDATE RESTRICT; immutable PKs.

- Indexes for common patient identifiers, encounters, clinical FKs, lab requests/status and prescription status.
- Simple name partial matching in V1; pg_trgm may be considered later.

Section 24 — SQLAlchemy & PostgreSQL Implementation

- PostgreSQL production database and SQLAlchemy ORM.
- Integer generated PKs.
- Application-controlled enumerations backed by PostgreSQL CHECK constraints.
- Proper timestamp/datetime fields.
- VARCHAR where sensible; TEXT for clinical narratives; JSONB for amendment content.
- No monetary fields in V1.

Section 25 — Flask Application Architecture

Modular Flask monolith using Application Factory and Blueprints: auth, desk, nursing, doctor, laboratory, pharmacy, admin, reports. Models are separated into files; services contain business logic; utils contain reusable helpers; Jinja2 templates and static assets are separated.

Section 26 — Authentication Workflow

- Phone login → identity established → role-specific dashboard.
- Password reset generates a new default password and displays it once.
- Deactivated accounts cannot log in.
- Logout terminates session.
- Authentication establishes identity; authorization determines permitted actions.

Section 27 — Workflow State Management

- Encounter is central; Desk creates OPEN encounter.
- Nursing does not close it.
- Doctor final submission closes it.
- Closed encounters cannot be manually reopened in V1.
- Late lab results may attach to closed encounters.
- Pharmacy receives only finalized prescriptions.

Section 28 — Role Dashboards

- Desk: Search Patient, Register New Patient, Create Encounter, Recent Patients, Today's Encounters.
- Nurse: Search Patient, Today's Encounters, Awaiting Nursing Assessment, My Nursing Drafts.
- Doctor: Search Patient, Today's Encounters, Awaiting Consultation, My Drafts, Pending Lab Results.
- Lab: Pending Requests, Search Patient/Request, Draft Results, Finalized Results.
- Pharmacy: Pending Prescriptions, Search Prescription/Patient, Doctor Communications, Dispensed Prescriptions.
- Admin: User Management, Investigation Catalogue, System Overview, System Search/View, Audit/Amendment History.

Section 29 — Reporting & Analytics

- Admin reporting dashboard with today/week/month/custom date filters.
- Patient/encounter, laboratory and pharmacy statistics.
- Normal/Abnormal/Critical lab counts.
- Simple charts and summary cards where useful.

- CSV and Excel exports using Pandas/openpyxl.
- No predictive analytics, surveillance or advanced BI in V1.

Section 30 — Backup & Recovery

- Daily PostgreSQL backups; retain most recent seven daily backups.
- Separate protected backup location with restricted access.
- Backups are not in Git or exposed through the application.
- Restore → verify integrity → resume service; restoration should be tested.
- No arbitrary hard-coded clinical retention period.
- V2 may add encrypted/off-site backups, PITR and formal DR.

Section 31 — Deployment & Environments

- Separate Development, Testing and Production environments.
- Initial development is local.
- Production requires Flask/Python, PostgreSQL, environment variables, HTTPS and persistent storage.
- Sensitive values are in environment variables; .env excluded from Git.
- Development/test/production databases are separate.
- Production updates follow backup → deploy → migrate → verify.
- No unnecessary Kubernetes/microservices/complex cloud architecture in V1.

Section 32 — Git, GitHub & Documentation

- Git used from the beginning; GitHub repository for source, migrations, tests and docs.
- No secrets, real patient data, production backups or .env files in Git.
- README covers purpose, features, stack, installation, usage, testing and architecture.
- Developer docs cover architecture, database, authentication, workflows, testing and deployment.
- User documentation is organized by role.
- PRD and implementation remain synchronized.

Section 33 — Testing & Quality Assurance

- Unit tests for Clinic ID, BMI, hashing, validation and permissions.
- Integration tests for patient/encounter/clinical relationships.
- End-to-end realistic patient journey tests.
- Security tests for role bypass, ownership and draft-result visibility.
- Regression testing after significant changes.
- User acceptance testing with synthetic patients and six test roles.
- Tools: pytest, Flask test client, SQLAlchemy test database and manual browser testing.
- Definition of Done = functionality + UI + database + validation + authorization + error handling + testing + documentation.

Section 34 — V1 Scope Boundary

- V1 includes authentication, patients, encounters, nursing, doctor, lab, pharmacy, admin, reports, security, audit/versioning, backup and deployment foundations.
- V2: allergies, blood group, genotype, richer problem lists, specimen workflow, structured lab ranges, inventory, partial dispensing, external messaging, advanced CDS, AI/ML prediction, 2FA, break-glass, advanced security monitoring, advanced BI/surveillance, microservices/React decomposition.

Section 35 — Acceptance Criteria & Definition of Done

End-to-end acceptance: Desk registers synthetic patient and generates Clinic ID → creates OPEN encounter → Nurse records/submits assessment → Doctor reviews and finalizes, automatically signing and closing encounter → Doctor requests investigation → Lab drafts/finalizes result, with draft hidden from doctor → Doctor sees finalized result → Doctor finalizes prescription → Pharmacy receives, communicates if needed and records dispensing → complete journey remains longitudinal and role-restricted.

Section 36 — Key Database Schemas

USERS
user_id PK; full_name; phone_number UNIQUE NOT NULL; password_hash NOT NULL; role_id FK; is_active; created_at; last_login_at

PATIENTS
patient_id PK; clinic_id UNIQUE NOT NULL; patient_category; full_name; date_of_birth; sex; phone_number; address; next_of_kin; student_id; department; faculty; level; staff_id; occupation; beneficiary_patient_id FK; created_at; is_active DEFAULT TRUE

ENCOUNTERS
encounter_id PK; patient_id FK; encounter_type; encounter_date; arrival_time; created_by FK; created_at; status

RECORD_AMENDMENTS
amendment_id PK; record_type; record_id; amended_by FK; amended_at; amendment_reason; previous_content JSONB; new_content JSONB

Section 37 — Implementation Sequence

Database → authentication → patient registration → encounters → nursing → doctor → laboratory → pharmacy → admin → reports → testing → deployment. Each module is built, tested and documented before proceeding.

Section 38 — Immediate Next Phase — Actual Build

- The requirements/design phase is complete through Section 52.
- Next phase is the real NUTM VITA-NOVA build.
- First: local development environment.
- Then: Flask Application Factory and project structure.
- Then: PostgreSQL/SQLAlchemy and migrations/models.
- Build module-by-module with tests alongside the code.

Section 39 — Locked Design Decisions — Quick Reference

- Exact name: NUTM VITA-NOVA.
- Modular Flask monolith; PostgreSQL + SQLAlchemy.
- Six roles exactly: Desk Officer, Nurse, Doctor, Laboratory Scientist, Pharmacist, Administrator.
- Phone number is username; one-hour inactivity timeout.
- Encounter is central; doctor final submission closes encounter.
- Late lab results attach without reopening.
- Only original clinical-record author may amend; history is permanent.
- Lab drafts hidden from doctors; Pharmacy cannot modify prescriptions.
- No queue numbers, patient photos/status, inventory, external notifications or advanced CDS in V1.
- Success toasts auto-dismiss after 2 seconds.
- CSV/Excel reporting; daily backups with seven-day retention; synthetic test data.

Section 40 — V2 / Future Expansion

- Allergies, blood group, genotype and richer problem lists.
- Specimen workflow and structured laboratory reference ranges/templates.
- Inventory, stock and partial dispensing.
- External messaging/notifications.
- Advanced CDS, drug interaction checking and AI/ML clinical prediction.
- 2FA, break-glass, advanced security monitoring.
- Encrypted/off-site backups, PITR and disaster recovery.
- Advanced BI, surveillance and PDF reporting.
- Frontend decomposition/microservices only if scale justifies them.

PRD STATUS

LOCKED THROUGH SECTION 52. This is the agreed V1 requirements/design baseline. Major changes should be treated as controlled scope changes and reviewed before implementation.

Next milestone: Begin actual coding and EMR construction.